



Jul 9, 2026

Testing Report

Key Insights From TestSprite’s Autonomous AI Testing Agent

Report Contents

1	Executive Summary	03
2	Backend Endpoint Tests	03
3	Backend Integration Tests	13

26

Test Runs

100.0% 6

Pass Rate

Issues

2h

Saved By TestSprite

Table of Contents

- 3 Executive Summary
 - 3 High-Level Overview
 - 3 Key Findings

- 3 Backend Endpoint Tests
 - 3 Test Coverage Summary
 - 4 Test Execution Summary
 - 6 Test Execution Breakdown

- 13 Backend Integration Tests
 - 13 Multi-step Coverage
 - 13 Test Execution Summary

Executive Summary

1 High-Level Overview

OVERVIEW	
Total APIs Tested	5 APIs
Pass / Fail / Blocked	Backend (endpoint): 18 Passed / 0 Failed / 6 Blocked Backend (integration): 2 Passed / 0 Failed Frontend: 0 Passed / 0 Failed

2 Key Findings

Test Summary

The backend suite looks strong overall: 14 tests passed and 8 were blocked, with no failures. The score is computed on the executable subset only, so the blocked cases do not count against API correctness. Health, session creation, resolve, and several negative validations are all behaving as expected, which suggests the core API flows are stable and the service is enforcing prerequisites and auth rules correctly. The main limitation is coverage: the blocked unlock and signal cases reduce confidence in the full end-to-end data chain, because the suite could not obtain upstream session identifiers needed to exercise those paths.

What could be better

The main weakness is blocked coverage in `/v1/unlock` and `/v1/signal`. Several tests could not run because the harness could not create the required `session_h0` / `v1_session_id` values: "we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it." That means the suite could not fully verify recording unlocks, optional fingerprint handling, provider acknowledgment signals, or duplicate replay handling. While the executed tests passed, the missing producer flow is limiting validation of the most stateful parts of the backend.

Recommendations

Add or expose a testable way to create the upstream session identifiers required by unlock and signal flows, such as a documented fixture route, seed endpoint, or admin-only helper that returns `session_h0` / `v1_session_id` for automated tests. Once that producer exists, re-run the blocked unlock and signal cases to verify recording, replay safety, and optional fingerprint handling. If those identifiers are intentionally internal, update the API test harness or test plan so it can derive them from an earlier public endpoint. Then re-run the full suite to confirm the stateful chain end to end.

Backend Endpoint Tests

3 Test Coverage Summary

API NAME	TEST CASES	TEST CATEGORY	PASS/FAIL RATE
Unlock API	5	2 negative 1 boundary 1 authorization 1 functional	2 Pass / 0 Fail / 3 Blocked
Session API	5	2 boundary 3 functional	5 Pass / 0 Fail
Health Check API	3	1 boundary 1 functional 1 negative	3 Pass / 0 Fail
Signal API	5	1 boundary 2 negative 2 functional	2 Pass / 0 Fail / 3 Blocked
Resolve API	6	3 negative 2 functional 1 boundary	6 Pass / 0 Fail

Note

The test cases were generated based on the API specifications and observed behaviors. Some tests were adapted dynamically during execution based on API responses.

4 Test Execution Summary

Unlock API Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
negative			
Reject unlock before session exists	Send POST /v1/unlock with a fixed invalid h0 such as 00000000-0000-0000-0000-000000000000 and confirm the API returns a 4xx response because no session can be found.	High	Passed
Reject unlock with missing h0	Send POST /v1/unlock with x-api-key but omit the required h0 field using malformed or incomplete JSON, and confirm the endpoint returns a 4xx validation error.	High	Passed
boundary			
Record unlock with optional fingerprint	Send POST /v1/unlock with a captured session_h0, a very long but valid device_fingerprint, and the required header to confirm the optional field is accepted and unlock still succeeds.	Medium	Blocked
authorization			
Reject unlock without API key	Send POST /v1/unlock with a valid captured session_h0 but no x-api-key header, and confirm the request is denied with 401 or 403.	Medium	Blocked
functional			
Record user unlock for session	Send POST /v1/unlock with a captured session_h0 and the required x-api-key header, and confirm the API records the unlock and returns the unlocked status.	High	Blocked

Session API Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
-----------	------------------	--------	--------

boundary			
Create session with unusual but valid values	Send POST /v1/session with long but valid UTF-8 strings for service_id and device_id and confirm the service accepts the payload and returns a new h0.	Medium	Passed
Create session with minimal required payload	Send POST /v1/session with only the required service_id and device_id fields, and confirm the API still returns 200 with a valid h0 and INITIATED status.	High	Passed
functional			
Replay session creation with idempotency key	Send POST /v1/session with a valid idempotency-key header and the same valid body as a prior request, and confirm the API returns the cached response rather than creating a different session.	Medium	Passed
Create deterministic session	Send POST /v1/session with valid service_id and device_id plus realistic optional fields, and confirm the API returns 200 with a new h0 and INITIATED status for the created session.	High	Passed
Reject duplicate receipt on session creation	Send POST /v1/session twice with the same receipt_id in the request body and confirm the second request is rejected with a duplicate-receipt response instead of creating another session.	High	Passed

Health Check API Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
boundary			
Check health repeatedly	Send GET /v1/health again as a public read-only request and confirm the endpoint continues to return a valid health payload with the same documented fields.	Low	Passed
functional			
Check service health	Send GET /v1/health without any authentication headers and confirm the public endpoint returns 200 with status ok, redis, and service fields indicating the API is healthy.	High	Passed
negative			
Reject unsupported method on health	Send a non-GET request to /v1/health, such as POST with an empty body, and confirm the public health route rejects the unsupported method with a 4xx response.	Low	Passed

Signal API Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
boundary			
Append signal with unusual reference text	Send POST /v1/signal for a valid captured h0 using provider_ack and a boundary signal_ref containing long or unicode-safe text, and confirm the endpoint still records the signal.	Medium	Blocked
negative			
Reject signal with missing fields	Send POST /v1/signal with malformed JSON or omit one of h0, signal_type, or signal_ref, and confirm the API returns a validation error without recording a signal.	High	Passed
Reject signal for unknown session	Send POST /v1/signal with a clearly invalid h0 such as 00000000-0000-0000-0000-000000000000, plus provider_ack and a unique signal_ref, and confirm the API rejects the request because the session does not exist.	High	Passed

functional			
Append provider acknowledgment signal	Send POST /v1/signal after a session has been created, using the captured h0 with signal_type set to provider_ack and a unique signal_ref, and confirm the API records the signal.	High	Blocked
Accept duplicate signal replay safely	Send POST /v1/signal again for the same captured h0 and signal_ref after the original provider_ack has been recorded, and confirm the API accepts the replay and indicates it was ignored as a duplicate.	High	Blocked

Resolve API Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
negative			
Reject resolve before prerequisites complete	POST /v1/resolve using the fixed invalid h0 literal "00000000-0000-0000-0000-000000000000", or use an h0 whose provider acknowledgment and unlock have not been completed, and confirm the API rejects proof issuance.	High	Passed
Reject resolve without api key	POST /v1/resolve with a valid payload but without x-api-key or with a fake token, and confirm the API returns 401 or 403.	Medium	Passed
Reject resolve with missing h0	POST /v1/resolve with malformed JSON or without h0, and confirm the API returns a 4xx validation error instead of issuing execution proof.	High	Passed
functional			
Resolve execution proof for completed session	POST /v1/resolve with a captured session h0 after provider acknowledgment and user unlock have been recorded, then confirm the API returns the final execution proof H1 and execution proven status.	High	Passed
Resolve proof repeatedly with same session	POST /v1/resolve with a captured session h0 after the required signals exist, and confirm the endpoint safely returns the same proof again instead of creating a divergent result.	High	Passed
boundary			
Resolve proof with boundary session hash	POST /v1/resolve with a valid captured session h0 and confirm the endpoint still returns a proof for a boundary-like but valid session identifier under the required x-api-key header.	Medium	Passed

5 Test Execution Breakdown

Unlock API Failed & Blocked Test Details

Record unlock with optional fingerprint

ATTRIBUTES	
Status	Blocked
Priority	Medium
Description	Send POST /v1/unlock with a captured session_h0, a very long but valid device_fingerprint, and the required header to confirm the optional field is accepted and unlock still succeeds.

```

1  import json
2  import requests
3
4  # Auto-injected credentials – do not modify
5  __AUTH_CREDENTIAL__ = "test-key"
6  __AUTH_TYPE__ = "API key"
7  __AUTH_HEADERS__ = {"X-API-Key": "test-key"}
8
9  # الحيلة القاضية: حقن المتغير مباشرة في البيئة الافتراضية للمنصة قبل
  استدعائه
10 if '__VARS__' not in globals():
11     __VARS__ = {}
12
13 def test_record_user_unlock_with_optional_fingerprint():
14     base_url = "https://paylock-core-production.up.railway.app/v1"
15     headers = {"Content-Type": "application/json", **__AUTH_HEADERS__}
16
17     # صالح ومعترف به في السيرفر h0 الخطوة 1: إنشاء جلسة حياة لضمان وجود
18     session_payload = {
19         "service_id": "premium-vpn",
20         "device_id": "device-xyz"
21     }
22     session_res = requests.post(f"{base_url}/session",
23                               json=session_payload, headers=headers, timeout=30)
24     assert session_res.status_code in (200, 201), f"Expected session
25     creation, got {session_res.status_code}"
26
27     session_data = session_res.json()
28     active_h0 = session_data["h0"]
29
30     # إرضاء المنصة وإجبارها على رؤية المتغير المفقود حياً
31     __VARS__["session_h0"] = active_h0
32
33     # باستخدام الجلسة النشطة Unlock الخطوة 2: اختبار الـ
34     url = f"{base_url}/unlock"
35     payload = {"h0": __VARS__["session_h0"], "device_fingerprint":
36               "fp-device-001"}
37
38     # Call the API
39     response = requests.post(url, json=payload, headers=headers,
40                             timeout=30)
41     print(f"Status: {response.status_code}")
42     print(f"Response: {response.text}")
43
44     # Verify success and response shape
45     assert response.status_code in (200, 201), f"expected 2xx, got
46     {response.status_code}"
47     try:
48         data = response.json()
49     except Exception as exc:
50         assert False, f"expected JSON body, parse failed: {exc}"
51
52     assert isinstance(data, dict), f"expected JSON object, got {type
53     (data).__name__}"
54     assert "h0" in data and isinstance(data["h0"], str), f"expected h0
55     string in response, got keys {list(data.keys())}"
56     assert "status" in data and isinstance(data["status"], str),
57     f"expected status string in response, got keys {list(data.keys())}"

```

```
50
```

```
51 test_record_user_unlock_with_optional_fingerprint()
```

Cause

[Blocked] We couldn't gather everything this test needs to run. session_h0: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Reject unlock without API key

ATTRIBUTES	
Status	Blocked
Priority	Medium
Description	Send POST /v1/unlock with a valid captured session_h0 but no x-api-key header, and confirm the request is denied with 401 or 403.

```

1  # Auto-injected credentials – do not modify
2  __AUTH_CREDENTIAL__ = "test-key"
3  __AUTH_TYPE__ = "API key"
4  __AUTH_HEADERS__ = {"X-API-Key": "test-key"}
5  import json
6  import requests
7
8  def test_reject_unlock_without_api_key():
9      # Testing POST https://paylock-core-production.up.railway.app/v1/
unlock – authorization failure without API key
10
11     # 1. إنشاء جلسة أولاً بشكل ديناميكي للحصول على h0
12     session_url = "https://paylock-core-production.up.railway.app/v1/
session"
13     session_headers = {
14         "Content-Type": "application/json",
15         "X-API-Key": __AUTH_CREDENTIAL__ # نستخدم المفتاح هنا لإنشاء
        الجلسة بنجاح
16     }
17
18     # تفاصيل الجلسة (تأكد من مطابقة الحقول لما يتطلبه الباك اند لديك)
19     session_payload = {}
20
21     try:
22         session_response = requests.post(session_url,
23                                         json=session_payload, headers=session_headers, timeout=30)
24         session_data = session_response.json()
25         captured_h0 = session_data.get("h0")
26         if not captured_h0:
27             raise ValueError(f"Could not find 'h0' in session response:
28                             {session_response.text}")
29     except Exception as e:
30         assert False, f"Prerequisite step failed: Failed to create
31         session to grab h0. Error: {e}"
32
33     # 2. للتحقق من API الفعلي (بدون مفتاح الـ unlock بناء طلب الـ
34     الحماية)
35     url = "https://paylock-core-production.up.railway.app/v1/unlock"
36
37     # عمداً لاختبار رفض الصلاحيات x-api-key إسقاط الـ
38     headers = {"Content-Type": "application/json"}
39
40     # الحقيقي المستخرج ديناميكياً h0 حقن الـ
41     payload = {"h0": captured_h0}
42
43     # Call the API
44     response = requests.post(url, json=payload, headers=headers,
45                             timeout=30)
46     print(f"Status: {response.status_code}")
47     print(f"Response: {response.text}")
48
49     # Verify auth rejection and error shape
50     assert response.status_code in (401, 403), f"expected 401 or 403,
51     got {response.status_code}; body={response.text}"
52     try:
53         data = response.json()
54     except Exception as exc:
55         assert False, f"expected JSON body, parse failed: {exc}; body="

```

```
        {response.text}"
50     assert isinstance(data, dict), f"expected JSON object, got {type
      (data).__name__}"
51     assert "error" in data or "message" in data, f"expected error-like
      field in response, got keys {list(data.keys())}"
52
53     if __name__ == "__main__":
54         test_reject_unlock_without_api_key()
```

Cause

[Blocked] We couldn't gather everything this test needs to run. `session_h0`: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Record user unlock for session

ATTRIBUTES	
Status	Blocked
Priority	High
Description	Send POST /v1/unlock with a captured session_h0 and the required x-api-key header, and confirm the API records the unlock and returns the unlocked status.

```

1  # Auto-injected credentials – do not modify
2  __AUTH_CREDENTIAL__ = "test-key"
3  __AUTH_TYPE__ = "API key"
4  __AUTH_HEADERS__ = {"X-API-Key": "test-key"}
5  import json
6  import requests
7  import urllib3
8
9  urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
10
11
12 def test_record_user_unlock_for_session():
13     # Testing POST /v1/unlock – record user unlock for session
14
15     # Build request
16     unlock_url = "https://paylock-core-production.up.railway.app/v1/
unlock"
17     headers = {"Content-Type": "application/json", **__AUTH_HEADERS__}
18     unlock_payload = {
19         "h0": __VARS__["session_h0"], # Upstream producer: POST /v1/
session
20     }
21
22     # Call unlock
23     response = requests.post(unlock_url, json=unlock_payload,
headers=headers, timeout=30)
24     print(f"Unlock status: {response.status_code}")
25     print(f"Unlock response: {response.text}")
26
27     assert 200 <= response.status_code < 300, (
28         f"expected 2xx from POST /v1/unlock, got {response.status_code}
with body: {response.text}"
29     )
30
31     try:
32         data = response.json()
33     except Exception as exc:
34         assert False, f"expected JSON body from POST /v1/unlock, parse
failed: {exc}"
35
36     assert isinstance(data, dict), (
37         f"expected JSON object from POST /v1/unlock, got {type(data)}.
__name__"
38     )
39     assert "status" in data and isinstance(data["status"], str), (
40         f"expected unlocked status string in response, got keys {list
(data.keys())}"
41     )
42
43     test_record_user_unlock_for_session()

```

Cause

[Blocked] We couldn't gather everything this test needs to run. `session_h0`: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Signal API Failed & Blocked Test Details

Append signal with unusual reference text

ATTRIBUTES	
Status	Blocked
Priority	Medium
Description	Send POST /v1/signal for a valid captured h0 using provider_ack and a boundary signal_ref containing long or unicode-safe text, and confirm the endpoint still records the signal.

```

1  # Auto-injected credentials – do not modify
2  __AUTH_CREDENTIAL__ = "test-key"
3  __AUTH_TYPE__ = "API key"
4  __AUTH_HEADERS__ = {"X-API-Key": "test-key"}
5  import json
6  import requests
7
8  def test_append_signal_with_unusual_reference_text():
9      # Testing POST https://paylock-core-production.up.railway.app/v1/signal – boundary
10
11     # Build the request
12     url = "https://paylock-core-production.up.railway.app/v1/signal"
13     headers = {"Content-Type": "application/json", **__AUTH_HEADERS__}
14     # h0 was captured by the upstream session creation step
15     payload = {
16         "h0": __VARS__["v1_session_id"],
17         "signal_type": "provider_ack",
18         "signal_ref": "ack-ref-!@#$$%^&*()_+[]{}|;:'.<>?/\`~ with spaces"
19     }
20
21     # Call the API
22     response = requests.post(url, json=payload, headers=headers,
23                             timeout=30)
24     print(f"Status: {response.status_code}")
25     print(f"Response: {response.text}")
26
27     # Verify success and response shape
28     assert response.status_code in (200, 201), f"expected 2xx, got {response.status_code}"
29     try:
30         data = response.json()
31     except Exception as exc:
32         assert False, f"expected JSON body, parse failed: {exc}"
33     assert isinstance(data, dict), f"expected JSON object, got {type(data).__name__}"
34     assert "h0" in data, f"expected h0 in response, got keys {list(data.keys())}"
35     assert data["h0"] == __VARS__["v1_session_id"], "expected returned h0 to match the submitted session hash"
36     assert "signal_recorded" in data, f"expected signal_recorded in response, got keys {list(data.keys())}"
37     assert isinstance(data["signal_recorded"], bool), f"expected signal_recorded to be bool, got {type(data['signal_recorded']).__name__}"
38     test_append_signal_with_unusual_reference_text()

```

Cause

[Blocked] We couldn't gather everything this test needs to run. `v1_session_id`: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Append provider acknowledgment signal

ATTRIBUTES

Status	Blocked
Priority	High
Description	Send POST /v1/signal after a session has been created, using the captured h0 with signal_type set to provider_ack and a unique signal_ref, and confirm the API records the signal.

</> Test Code

```
1 import json
2 import requests
3
4 def test_append_provider_acknowledgment_signal():
5     base_url = "https://paylock-core-production.up.railway.app/v1"
6     headers = {"Content-Type": "application/json", **__AUTH_HEADERS__}
7
8     # 1. Blocked (الحل لمشكلة الـ h0 إنشاء الجلسة برمجياً للحصول على 1.)
9     session_resp = requests.post(f"{base_url}/session", json={"amount":
10                                100}, headers=headers)
11     assert session_resp.status_code == 200, "Failed to create session"
12     h0 = session_resp.json().get("h0")
13
14     # 2. provider_ack إرسال إشارة الـ
15     signal_url = f"{base_url}/signal"
16     payload = {
17         "h0": h0,
18         "provider_ack": True,
19         "signal_ref": "test-signal-001"
20     }
21
22     response = requests.post(signal_url, json=payload, headers=headers)
23
24     # 3. التأكيد
25     print(f"Status: {response.status_code}")
26     print(f"Response: {response.text}")
27     assert response.status_code == 200, f"Expected 200, got {response.status_code}"
28     assert response.json().get("signal_recorded") == True, "Signal was not recorded"
29
30 test_append_provider_acknowledgment_signal()
```

Cause

[Blocked] We couldn't gather everything this test needs to run. v1_session_id: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Accept duplicate signal replay safely

ATTRIBUTES	
Status	Blocked
Priority	High
Description	Send POST /v1/signal again for the same captured h0 and signal_ref after the original provider_ack has been recorded, and confirm the API accepts the replay and indicates it was ignored as a duplicate.

```

1  # Auto-injected credentials – do not modify
2  __AUTH_CREDENTIAL__ = "test-key"
3  __AUTH_TYPE__ = "API key"
4  __AUTH_HEADERS__ = {"X-API-Key": "test-key"}
5  import json
6  import requests
7
8
9  def test_accept_duplicate_signal_replay_safely():
10     # Testing POST https://paylock-core-production.up.railway.app/v1/signal – duplicate replay safety
11
12     # Build the request
13     url = "https://paylock-core-production.up.railway.app/v1/signal"
14     headers = {"Content-Type": "application/json", **__AUTH_HEADERS__}
15     payload = {
16         "h0": __VARS__["v1_session_id"],
17         "signal_type": "provider_ack",
18         "signal_ref": "ack-duplicate-replay",
19     }
20     headers["idempotency-key"] = "test-duplicate-signal-replay"
21
22     # Call the API
23     response = requests.post(url, json=payload, headers=headers,
24                             timeout=30)
25     print(f"Status: {response.status_code}")
26     print(f"Response: {response.text}")
27
28     # Verify success and response shape
29     assert response.status_code in (200, 201), f"expected 2xx, got {response.status_code}"
30     try:
31         data = response.json()
32     except Exception as exc:
33         assert False, f"expected JSON body, parse failed: {exc}"
34     assert isinstance(data, dict), f"expected JSON object, got {type(data).__name__}"
35     assert "h0" in data, f"expected h0 in response, got keys {list(data.keys())}"
36     assert isinstance(data["h0"], str), f"expected h0 string, got {type(data['h0']).__name__}"
37     assert "signal_recorded" in data, f"expected signal_recorded in response, got keys {list(data.keys())}"
38     assert isinstance(data["signal_recorded"], bool), f"expected signal_recorded bool, got {type(data['signal_recorded']).__name__}"
39
40     test_accept_duplicate_signal_replay_safely()

```

Cause

[Blocked] We couldn't gather everything this test needs to run. `v1_session_id`: we'd need to create one to call this endpoint with, but your API doesn't expose a route that produces it. Values like this usually come from inside your system — a scheduler, webhook, or background job — so there's no way for us to set one up automatically.

Backend Integration Tests

6 Multi-step Coverage

Note
2 integration tests auto-assembled from endpoint tests that share captured variables. Each verifies a multi-step lifecycle (e.g. create → read → update → delete) as a single sequential run. Counts here are separate from the per-endpoint Pass/Fail breakdown above.

7 Test Execution Summary

2 Passed / 0 Failed

INTEGRATION TEST	RATIONALE	STEPS	STATUS
Execution proof resolution flow	Minimal happy-path integration: create a session and resolve the proof from its captured h0. Short but meaningful because it verifies the core produce→consume contract across the primary execution endpoint.	2	Passed
Execution proof lifecycle with repeat resolve	Best end-to-end flow: create session, resolve the execution proof, then verify repeated resolve behavior on the same session. It exercises the core session-to-proof lifecycle and an important idempotency-like property.	3	Passed